Arena object inspector

Chapter 3 · Practical Lab 3 · Runtime types and virtual dispatch

A game editor contains different kinds of objects. You can ask every object to report its state, but sometimes the editor also needs to know whether an object is a Guard, an EliteGuard, or something else. You will build a small inspector that asks those questions safely.

You will write the constructors, member functions, checks and loops yourself. I will give you the class interfaces, the rules each function must follow, short hints and expected console results. Work through one small task at a time.

Duration: **120 minutes** · **C++17** · **Visual Studio 2026** · **Create an Empty Project**

> **How to work.** Read the problem, make a prediction, write just the requested code, build and run, then compare your console output with the published result. Keep one `main()` throughout the practical.

Before you begin

You should already be comfortable with classes, public inheritance, constructors, virtual member functions, `override`, `const`, base pointers and small loops. In Visual Studio you will use Platform Toolset **v145**, **ISO C++17**, **/W4**, Debug and x64. This practical also needs **RTTI (/GR)** and C++ exceptions **(/EHsc)** enabled.

What you will practise

- Use a virtual base operation without checking concrete types unnecessarily.
- Use `dynamic_cast` to obtain a checked Guard view, and handle a failed cast.
- Use `typeid` to distinguish the pointer type from the exact type of a live object.
- Explain why a `Guard`-compatible view and an *exact* `Guard` are different tests.
- Inspect a small array containing live objects and null slots, without allocating or changing anything.
- Recognise a missing virtual-function definition as a possible linker problem.

Not required

No graphics or game engine, `new`/`delete`, raw memory inspection, vtable manipulation, compiler-specific class names, pointer arithmetic, performance benchmark, or student-written test framework. You will use local objects and the published console output to check your code.

The problem

The arena has an ordinary **Guard** (60 health), an **EliteGuard** (90 health and a 15-point bonus), and a **SupplyCrate** (4 items). They all have one shared action: `report()`. The editor can call that action through the `ArenaObject` interface without caring about the exact type.

When the editor needs a Guard-specific value, it should first check whether that view is valid. When it needs exactly a Guard, it should exclude EliteGuard even though EliteGuard inherits from Guard. Your inspector will demonstrate both decisions.

Files you will create

File	Purpose
Main.cpp	Create the objects, call the inspector and print labelled results.
ArenaObjects.h	Declare the abstract base and three concrete arena object types.
ArenaObjects.cpp	Implement constructors, reports and read-only queries.
Inspector.h	Declare the type-checking and selection functions.
Inspector.cpp	Write the checked casts, exact-type queries and bounded loops.

Your two-hour route

Stage	What you will do	Time
01	Create the Visual Studio project from scratch	10 min
02	Define arena objects and write their behaviour	30 min
03	Obtain a safe Guard view using a pointer cast	17 min
04	Ask for the exact runtime type	15 min
05	Handle a checked reference-cast failure	12 min
06	Count compatible and exact matches in a selection	22 min
07	Repair one deliberately missing virtual definition	6 min
08	Finish your console report and evidence record	8 min

01 · Create the project from scratch

In Visual Studio choose **Create a new project → Empty Project (C++)**. Name the project and solution `ArenaObjectInspectorLab`. You are creating the project yourself; there is no starter solution to import.

Select Debug and x64. In **Project → Properties** set **Platform Toolset = v145**, **C++ Language Standard = ISO C++17**, **Warning Level = Level 4 (/W4)**, **Enable Run-Time Type Information = Yes (/GR)** and **Enable C++ Exceptions = Yes (/EHsc)**. Apply these settings to All Configurations where appropriate. Do not use precompiled headers for this small project.

Under Source Files add `Main.cpp`. Type the following starter and build it.

```
#include <iostream>
#include <typeinfo>

int main()
{
    std::cout << std::boolalpha;
    std::cout << "Arena object inspector\n";
    return 0;
}
```

Run without debugging (Ctrl+F5). Your first output should be:

```
Arena object inspector
```

Next create the header `ArenaObjects.h` and the source file `ArenaObjects.cpp`. You will add `Inspector.h` and `Inspector.cpp` later. Leave exactly one `main()` in the project.

> **Checkpoint 01.** The empty project builds and prints its heading. If it does not, fix the project setup before adding class code.

02 · Define three kinds of arena object

Begin with a common interface. The base class says that every arena object must support a read-only `report()` function. That function is **pure virtual**, so you cannot construct an `ArenaObject` directly.

Type this first part into `ArenaObjects.h`:

```
#pragma once

class ArenaObject
{
public:
    virtual ~ArenaObject() = default;
    virtual void report() const = 0;
};

class Guard : public ArenaObject
{
public:
    explicit Guard(int t_health);
    void report() const override;
    int health() const;
private:
    int m_health;
};
```

Now add two more class declarations **below Guard in the same header**. Use this small list of requirements to write the class definitions yourself. You are building on the class syntax from previous practicals.

Class	Required declaration
`EliteGuard`	Derive publicly from `Guard`. Declare `EliteGuard(int t_health, int t_bonus)`, `void report() const override`, `int bonus() const` and private `int m_bonus`.
`SupplyCrate`	Derive publicly from `ArenaObject`. Declare `explicit SupplyCrate(int t_items)`, `void report() const override`, `int items() const` and private `int m_items`.

Hint. A derived constructor initialises its direct base before its own members. An EliteGuard constructor can begin with `: Guard{t_health}, m_bonus{t_bonus}`. Use `health()` to read the inherited private health value; do not access `Guard::m_health` directly.

Write the definitions

Open `ArenaObjects.cpp` and include `ArenaObjects.h` first, then `<iostream>`. Write the function bodies below. Use the `ClassName::functionName` form and remember the trailing `const` on read-only functions.

Function	What it should do
`Guard::Guard(int)`	Store the supplied health in `m_health` using a member initialiser.
`Guard::health() const`	Return the stored health.
`Guard::report() const`	Print `Guard health: ` followed by the health.
`EliteGuard::EliteGuard(int,int)`	Initialise `Guard` with health and initialise the bonus.
`EliteGuard::bonus() const`	Return the bonus.
`EliteGuard::report() const`	Print the health through `health()` and the bonus.
`SupplyCrate::SupplyCrate(int)`	Initialise the item count.
`SupplyCrate::items() const`	Return the stored item count.
`SupplyCrate::report() const`	Print `Supply crate items: ` followed by the item count.

Call the shared operation

In `Main.cpp`, include `ArenaObjects.h`. Below the heading, create the following objects and a **borrowed** base pointer. Use `selected->report()` three times: once for each object. Add the output section title `Shared behaviour` yourself.

```
Guard guard{60};
EliteGuard elite{90, 15};
SupplyCrate crate{4};

ArenaObject const* selected{&guard};
// Report guard, then point selected to elite and report.
// Finally point selected to crate and report.
```

When the three reports are correct, your first main checkpoint is:

```
Shared behaviour
Guard health: 60
Elite guard health: 90, bonus: 15
Supply crate items: 4
```

Checkpoint 02. All three concrete objects compile and report through the same `ArenaObject` pointer. The pointer does not create or own extra objects.

03 · Obtain a checked Guard view

The editor can report any arena object. However, it cannot ask every `ArenaObject` for health because `health()` is declared in `Guard`, not in the base. We need a checked, type-specific view before calling it.

Create `Inspector.h` with these declarations. Notice that the header needs only a forward declaration, because its function parameters use pointers or references rather than storing an `ArenaObject` by value.

```
#pragma once

class ArenaObject;

int readGuardHealth(ArenaObject const* t_object);
bool isExactGuard(ArenaObject const* t_object);
int requireGuardHealth(ArenaObject const& t_object);
int countGuardViews(ArenaObject const* const* t_objects,
                    int t_count);
int countExactGuards(ArenaObject const* const* t_objects,
                     int t_count);
```

Create `Inspector.cpp` and include `Inspector.h`, `ArenaObjects.h` and `<typeinfo>`. Define **only** `readGuardHealth()` first, using the following rules:

Step	Required behaviour
1	Ask for a read-only `Guard` pointer using `dynamic_cast`.
2	If the cast returns `nullptr`, return `-1` immediately.
3	Otherwise, call `health()` through the checked pointer and return it.

The most important fragment is the cast itself. You must write the `if` and the return statements:

```
Guard const* pGuard{dynamic_cast<Guard const*>(t_object)};
```

In `Main.cpp`, include `Inspector.h`. Under a heading `Checked guard views`, call the new function with `&guard`, `&elite`, `&crate` and `nullptr`, then print the four returned values using the labels below.

```
Checked guard views
Guard: 60
Elite guard: 90
Crate: -1
None: -1
```

An EliteGuard contains a Guard base part, so it **can** be viewed as Guard. A SupplyCrate cannot. A null input also yields a null pointer-cast result. Do not dereference a failed cast.

> **Checkpoint 03.** The pointer cast succeeds for both Guard and EliteGuard. It fails safely for SupplyCrate and null. No object has been copied or allocated.

04 · Ask for the exact type

Sometimes the editor needs **exactly** an ordinary Guard, not an EliteGuard. That is a different question from asking whether a Guard view is available.

In `Inspector.cpp`, define `isExactGuard(ArenaObject const* t_object)`. First handle a null pointer by returning `false`. For a live object, compare its runtime type with `Guard` using `typeid`. Remember that `typeid(*t_object)` asks about the polymorphic object, while `typeid(t_object)` only describes the pointer expression.

```
// Fragment, not the complete function:
typeid(*t_object) == typeid(Guard)
```

In `Main.cpp`, add the heading `Exact type checks`. Print `isExactGuard(&guard)`, `isExactGuard(&elite)` and `isExactGuard(&crate)`. Then ask two further questions directly with `typeid`: whether a base pointer currently referring to elite identifies an exact `EliteGuard` object, and whether the pointer variable itself has type `ArenaObject const*`.

```
Exact type checks
Guard is exact Guard: true
Elite is exact Guard: false
Elite is exact EliteGuard: true
Crate is exact Guard: false
Pointer is ArenaObject const*: true
```

> **Think.** Why can `dynamic_cast<Guard const*>(&elite)` succeed even though `typeid(elite) == typeid(Guard)` is false? One test asks for a usable base view; the other asks for exact type identity.

> **Checkpoint 04.** Both kinds of question give their published results. Do not compare `typeid(...).name()` with a fixed class-name string; that text may vary across compilers.

05 · Try a checked reference instead

A pointer cast can return `nullptr`. A checked reference cast has no null result. When it cannot obtain the requested derived reference, C++ throws `std::bad_cast`.

In `Inspector.cpp`, define `requireGuardHealth(ArenaObject const& t_object)`. Obtain a `Guard const&` using `dynamic_cast<Guard const&>(t_object)` and return `health()` through that reference. Do not catch the error inside the helper: allow the caller to handle it.

In `Main.cpp`, add `Reference casts`. Print the result for `guard`, then call the same function with `crate` inside `try`. Add `catch (std::bad_cast const&)` and print `not a Guard` when the conversion fails. Use the label `Crate reference:` immediately before the checked call.

```
Reference casts
Guard reference: 60
Crate reference: not a Guard
```

> **Hint.** Only the conversion fails; `crate` is still the same live SupplyCrate object. Catching `std::bad_cast const&` is different from testing a pointer result against `nullptr`.

06 · Inspect a small mixed selection

The editor can select several objects at once. Instead of checking names or making another copy of each object, keep pointers to the existing locals. A selection may also include an empty (`nullptr`) slot.

In `Main.cpp`, create this four-slot selection:

```
ArenaObject const* slots[]{&guard, &elite, &crate, nullptr};
```

Now finish the two declarations already present in `Inspector.h`. Write the function bodies in `Inspector.cpp` yourself. The important difference is **which matching rule** each function uses.

Input rule	Required result for BOTH functions
Count less than 0 or greater than 4	Return `-1` immediately. Do not read the array.
Count is exactly 0	Return `0`, even if the array pointer is null.
Positive count but null array pointer	Return `-1`, without reading the array.
Null element in a real array	Skip it without making an object-type query.
Non-null live object	Count it only when it satisfies the function's matching rule.

For `countGuardViews()`, scan the slots from first to last and count successful `dynamic_cast<Guard const*>` results. For `countExactGuards()`, repeat the same input checks but count only non-null slots whose `typeid(*object)` equals `typeid(Guard)`. Use a straightforward `for` loop; you have already practised this in earlier labs.

```
// A useful loop fragment; write the rest of both functions.
for (int index{0}; index < t_count; ++index)
{
    // Check this slot and update your local count.
}
```

Your final four selection calls should print:

```
Selection
Guard-compatible: 2
Exact guards: 1
Empty selection: 0
Missing storage: -1
```

The function assumes a positive count points to at least that many actual array slots, and that all non-null slots point to live objects. A null check cannot prove that an arbitrary non-null pointer has not expired.

> **Checkpoint 06.** Both functions return 2 and 1 for the same mixed selection. They also handle count 0 and a missing array pointer without indexing invalid storage.

07 · Diagnose one missing virtual definition

You have been declaring member functions in headers and defining them in `.cpp` files. Here is a quick way to see why both pieces are required. Save a working version before making the temporary change.

In `ArenaObjects.cpp`, temporarily comment out **the complete `EliteGuard::report() const` definition**, including its function header and body. Keep the declaration in `ArenaObjects.h`. Now select **Build → Build Solution**. Record whether the error is reported during compilation or linking. The linker may mention `EliteGuard::report` or a related virtual-table symbol.

Restore the complete function definition, build again and run your usual output. Do not run a previously built executable after the failed build. Do not remove `override` to hide the missing body.

> **Checkpoint 07.** The original build succeeds again. Explain in one sentence why a valid declaration did not provide the function definition that linking required.

08 · Put Main.cpp into its final form

Keep the calls you have already written. You do not need to rewrite them from a completed driver program. Use the outline below to put the labelled sections in one sensible order and remove duplicated temporary statements.

```
int main()
{
    // Set std::boolalpha; print the lab heading.
    Guard guard{60};
    EliteGuard elite{90, 15};
    SupplyCrate crate{4};
    ArenaObject const* selected{&guard};

    // Shared behaviour: report all three objects.
    // Checked views: print four readGuardHealth results.
    // Exact checks: use isExactGuard and typeid.
    // Reference casts: handle the failed crate cast.
    ArenaObject const* slots[]{&guard, &elite, &crate, nullptr};
    // Selection: compatible count, exact count and boundaries.
    return 0;
}
```

Keep the appropriate `#include` lines at the top of `Main.cpp`; this is a **short outline**, not a file to paste over your earlier work. Print the headings, labels and expressions yourself, using the complete expected output below as the guide.

Final expected console output

```
Arena object inspector

Shared behaviour
Guard health: 60
Elite guard health: 90, bonus: 15
Supply crate items: 4

Checked guard views
Guard: 60
Elite guard: 90
Crate: -1
None: -1

Exact type checks
Guard is exact Guard: true
Elite is exact Guard: false
Elite is exact EliteGuard: true
Crate is exact Guard: false
Pointer is ArenaObject const*: true

Reference casts
Guard reference: 60
Crate reference: not a Guard

Selection
Guard-compatible: 2
Exact guards: 1
Empty selection: 0
Missing storage: -1
```

Checkpoint 08. Your project builds in Debug x64, runs without accessing invalid objects, and produces the same labelled values as the expected output. Retain your source files and the short evidence record.

If something is not working

Symptom	Check this first
An override does not compile	Check the exact method name, argument list and trailing `const`.
A virtual symbol is unresolved	Look for the matching `.cpp` definition and confirm that file is in the project.
The EliteGuard health line will not compile	Use the public inherited `health()` query, not Guard's private `m_health`.
Both matching counts are 2	An EliteGuard is compatible with Guard, but not exactly Guard.
`typeid` reports the pointer type	You may be querying `selected` instead of the live object `*selected`.
A null slot causes a failure	Check the slot before using `typeid(*slot)` or any member function.

Final check

- I created my Visual Studio project from scratch and added all five required files.
- My concrete classes implement the exact required `report() const` operation.
- My checked pointer cast does not dereference a failed result.
- My exact-type query distinguishes Guard from EliteGuard.
- My reference cast handles a wrong-type result with `std::bad_cast`.
- My selection functions validate counts and ignore null slots safely.
- I restored the deliberate missing-definition error and my final output matches.

Optional stretch · Observe size, not object layout

If you finish early, create two small structures with two `int` data members. Give one of them a virtual destructor. Print `sizeof` for each type using your installed compiler. Record the values and explain why the result is **not** a portable measure of an entire vtable or a safe field offset. Do not cast the object to an array of integers, and do not attempt raw memory writes.

An implementation may use a hidden virtual pointer (vptr) and a table of function entries, but the exact layout is not prescribed by C++. Your task is to understand the selected behaviour, not to reverse-engineer the compiler's private memory structure.

Before finishing, save your code, complete the matching evidence template and submit only the files requested by your lecturer.